New Horizons
Centro de Capacitación

Práctica: Generación de Tokens JWT y Servicio de Autenticación en C#

Paso a paso — Configurar appsettings, SymmetricSecurityKey, claims de usuario y JwtSecurityTokenHandler

Curso	Desarrollo Full Stack con C# y Angular
Sesión	Sesión 13 — Seguridad y Autenticación Backend con JWT en ASP.NET Core
Docente	Edwin Barrientos
Duración estimada	75–90 minutos
Nivel	Básico (guía principal de la sesión)
Entorno	.NET 8 SDK, Visual Studio 2022 / VS Code, Angular 18 Standalone, Bootstrap 5

1. Objetivos de la Práctica

- Configurar los parámetros del emisor, audiencia y clave secreta JWT en appsettings.json.
- Implementar IAuthService en la capa Application para encapsular la lógica de autenticación.
- Construir el generador de tokens emitiendo claims estándar (sub, email) y de negocio (rol, id).
- Firmar el token utilizando el algoritmo simétrico HMAC-SHA256 con claves seguras.
- Verificar la estructura del token generado utilizando la herramienta online jwt.io.

2. Desarrollo Paso a Paso

Paso 1 — Configurar Parámetros JWT en appsettings.json

Definimos la clave secreta de al menos 256 bits, el emisor y la audiencia dentro de appsettings.json de la API:

src/SolicitudesApp.Api/appsettings.json

```
{
  "Jwt": {
    "Key": "ClaveSecretaSuperSeguraParaFirmarTokensNewHorizons2026FullStack!",
    "Issuer": "https://solicitudesapp.empresa.com",
    "Audience": "https://solicitudesapp-frontend.empresa.com",
    "DuracionHoras": 8
  }
}
```

Paso 2 — Crear el Contrato IAuthService y DTOs de Autenticación

Definimos los modelos de entrada y salida para el proceso de inicio de sesión en Application/DTOs/Auth:

src/SolicitudesApp.Application/DTOs/AuthDtos.cs

```
public record LoginRequestDto(string Email, string Password);

public record AuthResponseDto(
    int Id,
    string Nombre,
    string Email,
    string Rol,
    string Token,
    DateTime Expiracion
);
```

Paso 3 — Implementar la Emisión de Tokens JWT

Construimos AuthService implementando la generación de credenciales y claims criptográficos:

src/SolicitudesApp.Infrastructure/Services/AuthService.cs

```
public class AuthService : IAuthService
{
    private readonly IConfiguration _config;
    public AuthService(IConfiguration config) => _config = config;

    public string GenerarToken(Usuario usuario)
    {
        var claims = new List<Claim>
        {
            new(ClaimTypes.NameIdentifier, usuario.Id.ToString()),
            new(ClaimTypes.Name, usuario.NombreCompleto),
            new(ClaimTypes.Email, usuario.Email),
            new(ClaimTypes.Role, usuario.Rol.Nombre)
        };

        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Key"]!));
        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

        var token = new JwtSecurityToken(
            issuer: _config["Jwt:Issuer"],
            audience: _config["Jwt:Audience"],
            claims: claims,
            expires: DateTime.UtcNow.AddHours(8),
            signingCredentials: creds
        );

        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}
```

Validación de la Práctica: Asegúrese de compilar y ejecutar las pruebas para validar que los endpoints y vistas operen conforme a los estándares corporativos solicitados.